

Advanced Flying Robots – Final Results

INDI Across Three Platforms: Standard, Thrust-Upgraded, Brushless CF21BL

Georg Wolnik

Multi Robot Systems Project
Mentors: Omar Elsayed, Khaled Wahba
Supervisor: Prof. Wolfgang Hönig

Final Presentation

Agenda

- ➊ Recap: planned trajectories + geometric vs. INDI (standard drone)
- ➋ Core result: figure-8 across all four modes
- ➌ Upgraded vs. brushless: oval, circle + kt sweeps
- ➍ Limitations & complications
- ➎ Summary

From the last progress presentation

- Minimum-snap polynomial trajectories; two timing modes: Mode 1 (k_t timing), Mode 3 (time redistribution).
- Differential flatness maps the planned trajectory to the controller feedforward (thrust, attitude, $\dot{\omega}_d$).
- INDI (Tal & Karaman 2020): incremental, sensor-based control law, benchmarked on standard CF2.1.

Motion Planning — The Idea

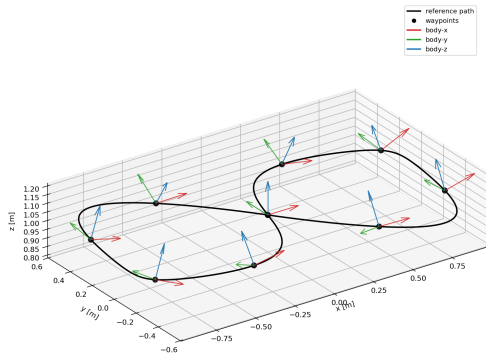
Core idea, in one sentence

Turn a handful of waypoints into **one smooth, closed-form path** — and get the drone's full attitude and thrust commands **for free** from that same path, instead of planning them separately.

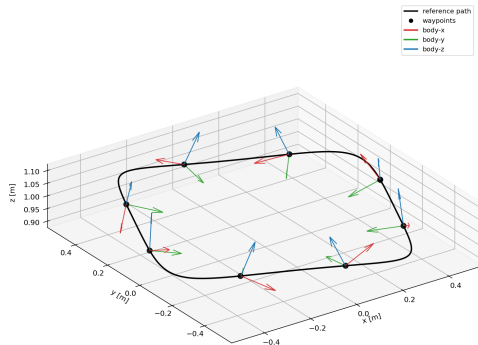
- **Problem:** waypoints alone are not a flyable trajectory — need something smooth, dynamically feasible, and exactly repeatable, not just “connect the dots.”
- **Minimum-snap polynomials** (Mellinger & Kumar 2011): fit a piecewise polynomial through the waypoints minimizing snap (4th derivative of position), continuous up to snap at every segment boundary — snap drives *required thrust and attitude rate*, so smoother snap means gentler, more achievable commands.
- **Differential flatness** (Mellinger & Kumar 2011; rotor-drag extension: Faessler et al. 2018): the quadrotor's full state — attitude, body rate, thrust — is recovered in *closed form* from the position path and its derivatives plus yaw. No attitude has to be authored by hand.
- **Full feedforward for every controller:** geometric *and* INDI both get thrust, attitude, body rate, and angular acceleration from the same path — enabling a fair, apples-to-apples comparison across controllers and platforms.

Planned Trajectories — Figure-8 / Circle / Corner

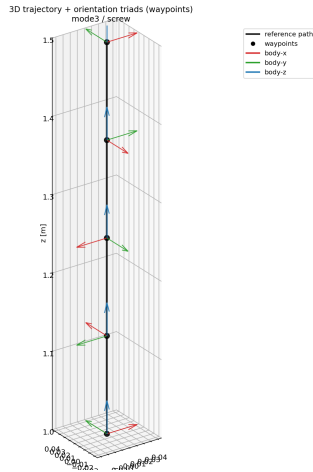
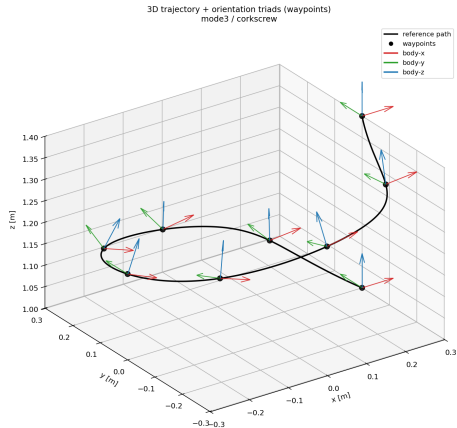
3D trajectory + orientation triads (waypoints)
model / figure8



3D trajectory + orientation triads (waypoints)
mode3 / circle

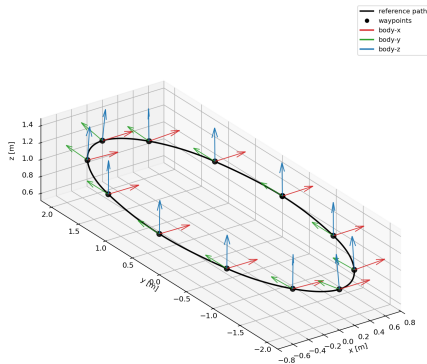


Planned Trajectories — Corkscrew / Screw

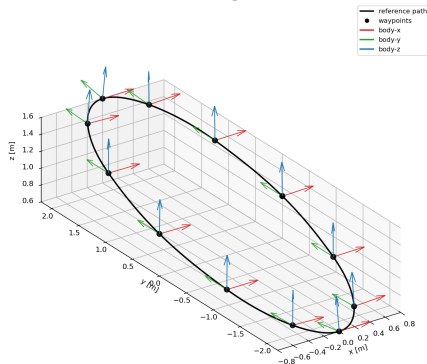


Planned Trajectories — Racing: Oval / Tilted Oval

3D trajectory + orientation triads (waypoints)
model / oval



3D trajectory + orientation triads (waypoints)
model / tilted_oval



INDI — Attitude Control via Incremental Sensor Feedback

Core idea, in one sentence

Don't compute the torque you need from scratch — **measure the torque the motors are already producing right now**, and command only the **small correction** needed to close the remaining gap to the desired angular acceleration.

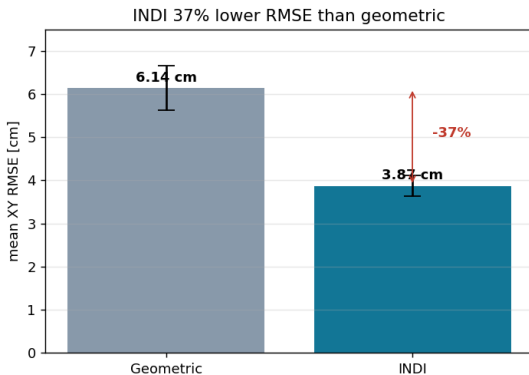
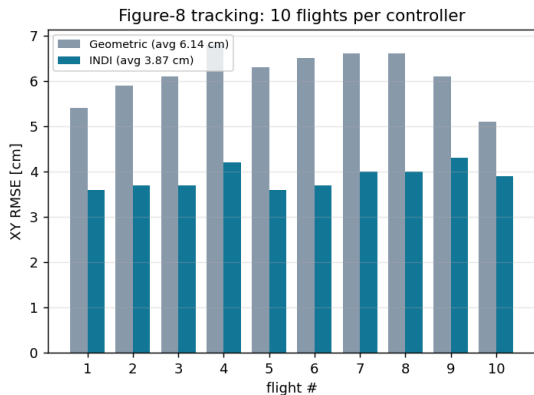
- **Classical (global) nonlinear dynamic inversion** computes the *entire* control input from a complete dynamics model every step — get the model wrong and the whole command is wrong.
- **INDI** (Tal & Karaman 2020) instead measures where the system already is and only asks the model for the small step from there to the target:

$$\underbrace{\tau}_{\text{command}} = \underbrace{\tau_{\text{current}}}_{\text{measured, RPM}^2} + \underbrace{J(\alpha_{\text{ref}} - \alpha_{\text{meas}})}_{\text{small correction, from model}}$$

(α_{meas} : measured from gyro; α_{ref} : desired angular acceleration.)

- Only that *small correction* needs the model (effectiveness J , thrust constants) — sensors answer “where am I now,” the model answers only “how much does a small extra input change things.”
- **Advantage over standard controllers**: unlike PID (reacts after the fact) or pure model-based control (only as accurate as its inertia model), INDI stays accurate even when that model is wrong — its largest gains landed on the one unidentified platform.

Geometric vs. INDI — Standard CF2.1, 10 Flights Each



37% lower RMSE with INDI, consistent with the Tal & Karaman reference paper.

Figure-8 — Four-Way Comparison, Mean-Closest Flight

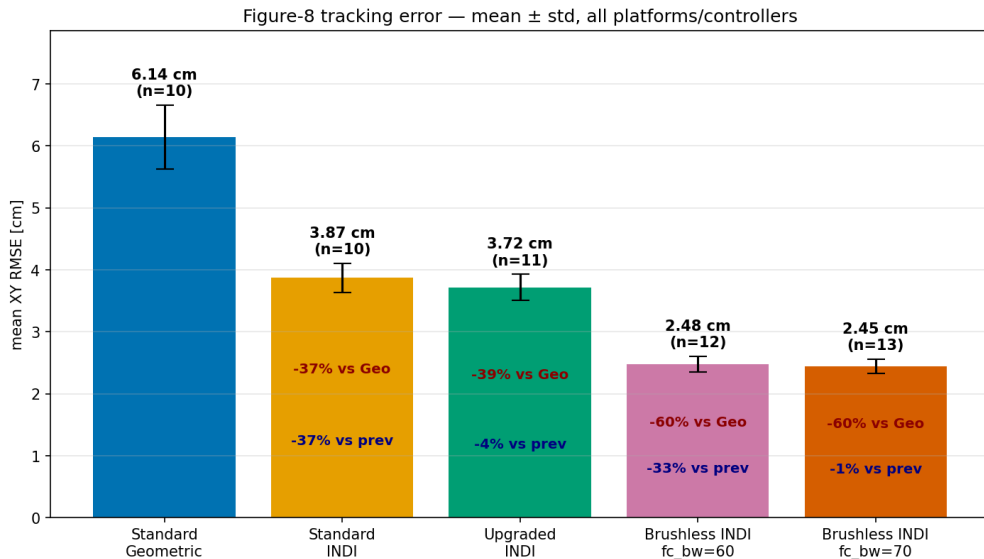


Figure-8 — Best Flight, All Four Modes: Overview

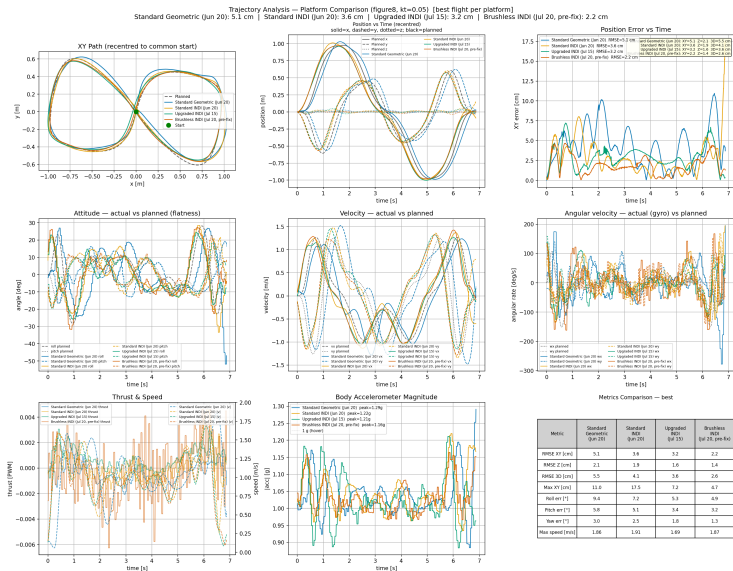


Figure-8 — Best Flight, All Four Modes: Per-Axis

Per-axis Tracking Subplots — Platform Comparison (figure8, kt=0.05) [best]

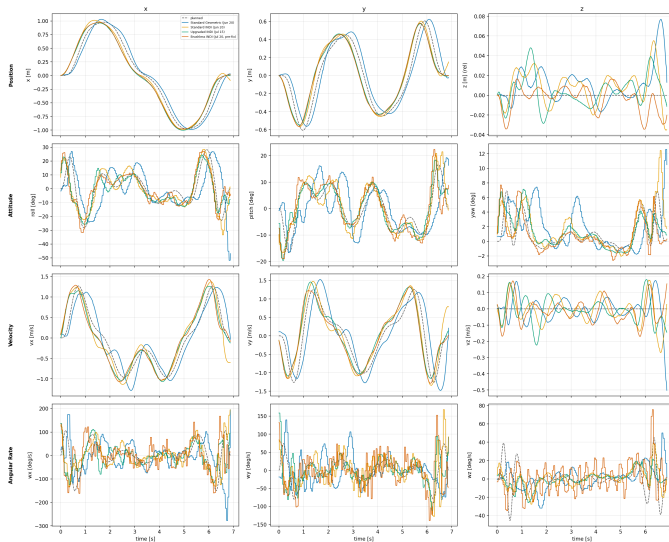


Figure-8 — Best Flight, All Four Modes: 3D Orientation

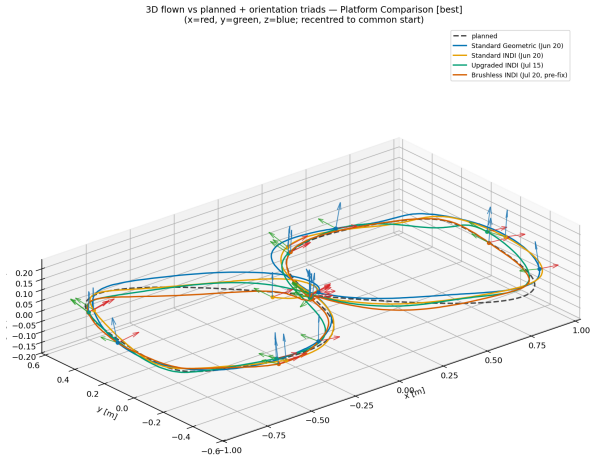
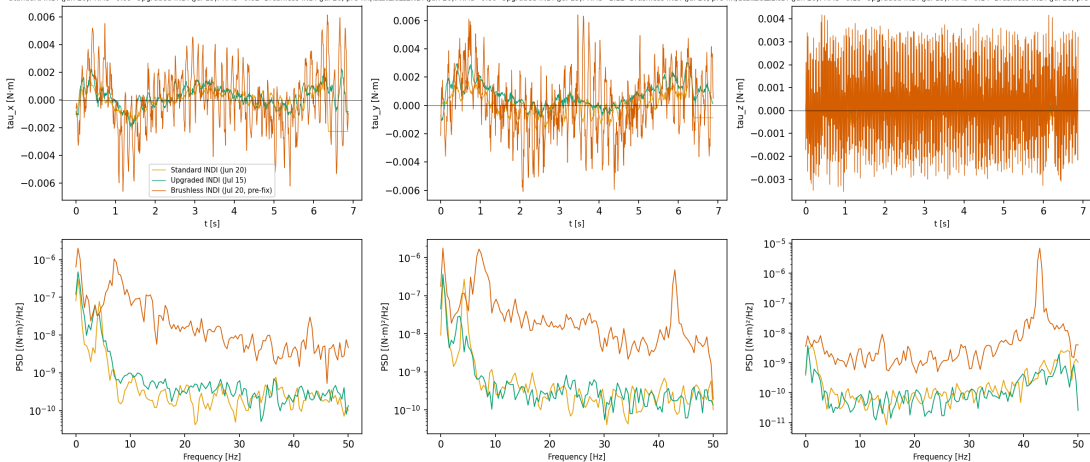


Figure-8 — Best Flight, All Four Modes: INDI Panel

INDI Torque Panel — Platform Comparison (figure8, kt=0.05) [best]

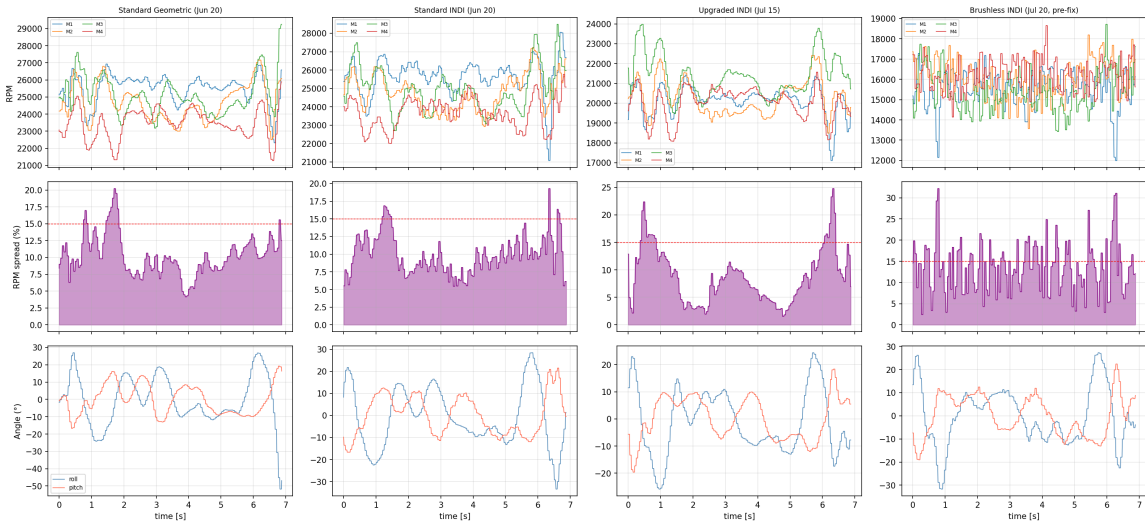
Standard INDI (Jun 20): RMS=0.89 Upgraded INDI (Jul 15): RMS=0.82 Brushless INDI (Jul 20, pre-fix): RMS=0.80 Standard INDI (Jun 20): RMS=0.80 Upgraded INDI (Jul 15): RMS=1.12 Brushless INDI (Jul 20, pre-fix): RMS=0.77 Standard INDI (Jun 20): RMS=0.19 Upgraded INDI (Jul 15): RMS=0.14 Brushless INDI (Jul 20, pre-fix): RMS=2.17



No INDI telemetry (skipped): Standard Geometric (Jun 20)

Figure-8 — Best Flight, All Four Modes: RPM Balance

RPM Balance — Platform Comparison (figure8, kt=0.05) [best]

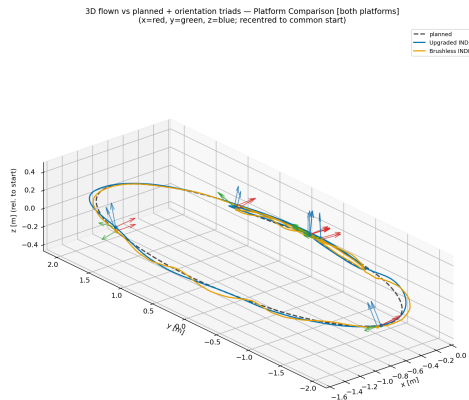


Oval kt Sweep — Both Platforms

kt	XY / Z RMSE		Roll / Pitch / Yaw err		Max speed [m/s]	
	Upgraded	Brushless	Upgraded	Brushless	Upgraded	Brushless
0.05	2.1 / 0.8 cm	2.3 / 0.5 cm	1.7/1.3/0.8°	2.2/2.9/0.9°	1.10	1.22
0.1	2.8 / 0.9 cm	1.3 / 0.7 cm	2.3/1.5/0.7°	3.1/1.6/0.3°	1.33	1.32
0.2	2.9 / 1.3 cm	2.5 / 1.1 cm	3.3/1.5/1.0°	2.6/1.3/0.4°	1.61	1.51
0.3	3.0 / 1.8 cm	2.8 / 1.0 cm	3.2/1.7/1.4°	2.9/1.5/0.5°	1.76	1.72
0.4	3.2 / 2.1 cm	3.2 / 1.4 cm	4.4/2.1/1.9°	10.5/6.1/2.0°	1.97	2.25
0.5	crash (5/5)	3.1 / 1.6 cm	—	5.5/2.7/1.4°	—	2.20
0.6	crash	not flown	—	—	—	—
0.7	not flown	crash	—	—	—	—

Working ceiling: upgraded ~kt 0.3–0.4, brushless ~kt 0.5–0.7.

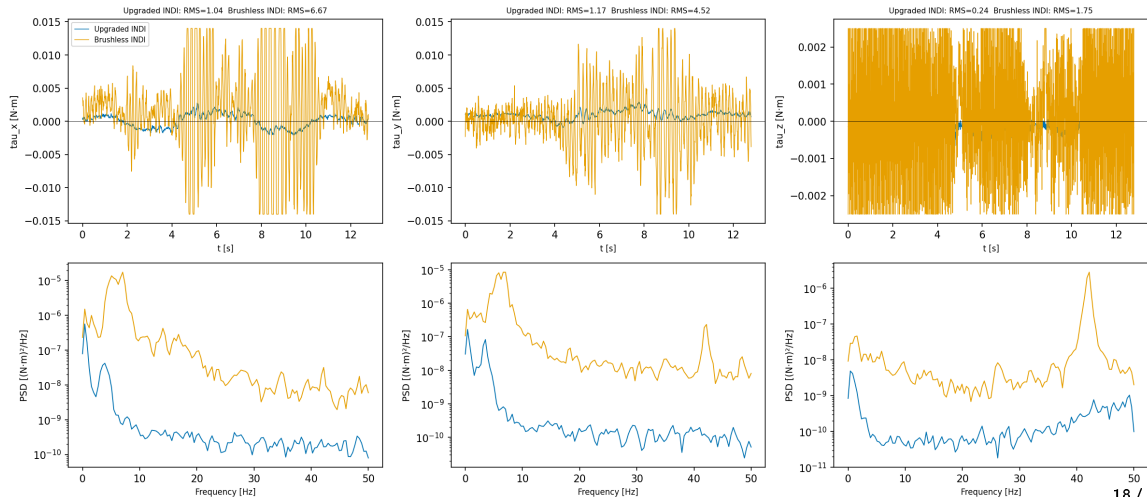
Oval $kt=0.4$ — Both Platforms: 3D Orientation



XY/Z RMSE: upgraded 3.2/2.1 cm, brushless 3.2/1.4 cm — comparable XY, brushless tighter on Z.

Oval kt=0.4 — Both Platforms: INDI Panel

INDI Torque Panel — Platform Comparison (oval, kt=0.4) [both platforms]

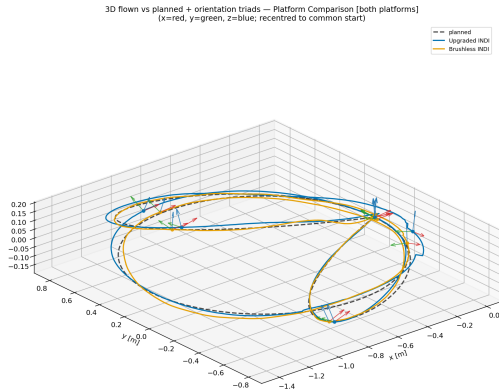


Circle kt Sweep — Both Platforms

kt	XY / Z RMSE		Roll / Pitch / Yaw err		Max speed [m/s]	
	Upgraded	Brushless	Upgraded	Brushless	Upgraded	Brushless
0.05	1.7 / 0.8 cm	1.9 / 0.6 cm	2.1/1.8/0.8°	1.9/1.4/0.6°	1.03	1.21
0.1	1.9 / 1.3 cm	1.7 / 0.6 cm	1.9/1.8/1.1°	4.2/3.9/0.7°	1.15	1.50
0.2	2.3 / 1.2 cm	6.1 / 1.8 cm	2.2/1.8/1.4°	14.5/13.2/3.6°	1.35	1.71
0.3	2.2 / 1.5 cm	1.4 / 0.8 cm	2.3/2.1/1.9°	3.1/2.7/0.6°	1.53	1.75
0.4	2.7 / 1.8 cm	4.2 / 1.4 cm	3.5/3.6/2.5°	10.0/9.3/2.4°	1.73	1.84
0.5	2.8 / 1.6 cm	3.7 / 1.2 cm	3.5/3.5/2.6°	7.9/7.5/2.0°	1.84	1.88
0.6	3.7 / 2.4 cm	1.9 / 1.0 cm	3.1/3.3/2.7°	4.7/4.9/1.5°	1.98	1.84
0.7	4.2 / 1.7 cm	2.4 / 1.2 cm	4.4/5.6/3.4°	7.3/6.8/2.3°	2.11	2.13
0.8	3.3 / 2.5 cm	crash	4.8/5.3/3.6°	—	2.37	—
0.9	crash	crash	—	—	—	—
1.0	3.0 / 3.6 cm	not flown	4.9/4.0/4.6°	—	2.48	—

Both platforms show the same pattern: a clean ceiling, then a marginal/inconsistent zone — not a sharp wall, not a platform-specific quirk.

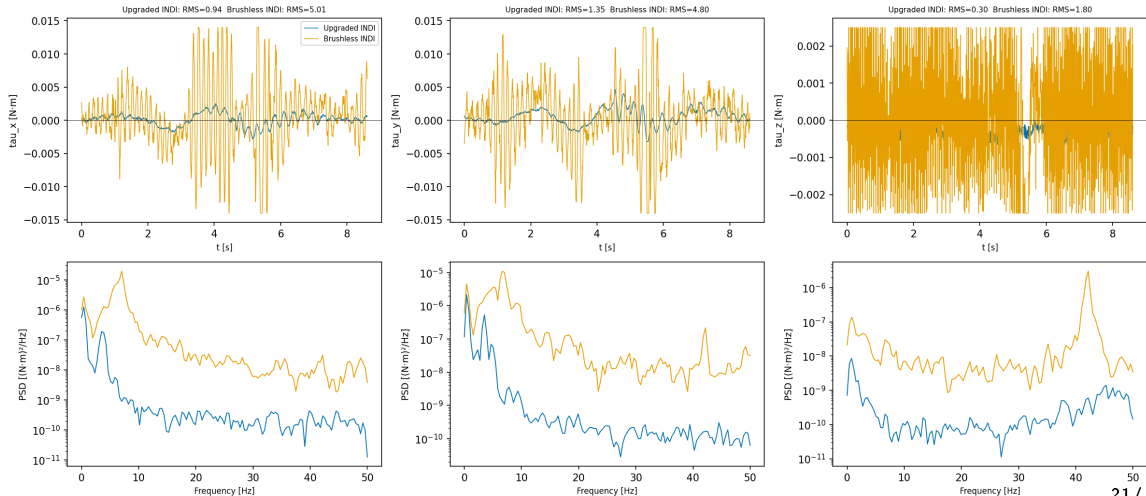
Circle $kt=0.7$ — Both Platforms: 3D Orientation



XY/Z RMSE: upgraded 4.2/1.7 cm, brushless 2.4/1.2 cm — brushless tighter on both axes here.

Circle kt=0.7 — Both Platforms: INDI Panel

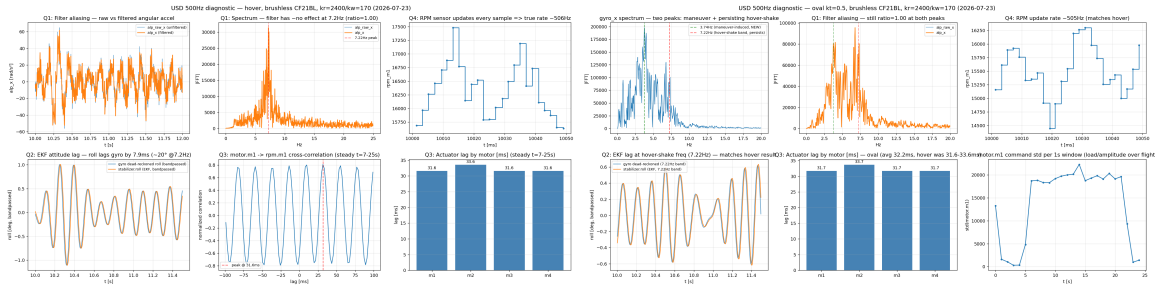
INDI Torque Panel — Platform Comparison (circle, kt=0.7) [both platforms]



The Brushless “Shaky Shaky” — What We Saw

What we saw

- Full INDI on brushless CF21BL: a self-sustained ~ 7 Hz oscillation, reproducible in hover *and* under maneuvering load — same mechanism throughout.
- **This is why brushless underperforms upgraded on some maneuvers** (e.g. crash ceiling on oval/circle at high kt) despite otherwise excellent figure-8 tracking — an unexpected, counterintuitive combination worth flagging explicitly.



Root Cause — A Gain-Driven, Low-Frequency Resonance, Not Noise

The key match

$\omega_n = \sqrt{k_r}/2\pi = 7.8 \text{ Hz}$ at $k_r = 2400$ — almost exactly the measured 7.22 Hz shake. The controller is oscillating near its *own* attitude-loop natural frequency. That makes this a **low-frequency, gain-driven** closed-loop resonance, not a high-frequency external noise/vibration source being poorly filtered.

What we tried	Result	Verdict
Filter (fc_bw=70 Hz)	alp_filt/alp_raw ratio = 1.00	dead end
Notch filter (0.25–20 Hz)	tracking worse in every stable case	dead end
Detuning kr (≈ 1500)	cascade timescale violation instead	dead end

- Why the noise-oriented fixes failed: both filter and notch target a signal problem; this is a gain problem. Attenuating the resonance in the error signal removes the stabilizing feedback constraining it, rather than removing noise — so it makes tracking worse, not better, exactly as observed.
- Why simply lowering k_r failed: the attitude loop cannot be detuned in isolation — it must stay meaningfully faster than the outer position loop, and lowering k_r alone breaks that separation before it fixes the resonance.
- Actuator lag (32 ms $\rightarrow$ 82° phase at 7.22 Hz) and EKF attitude lag (7.9 ms $\rightarrow$ 20°) are both measured and consistent with the picture, but *secondary* to the gain match above — most likely the real lever is the gain/ bandwidth relationship itself, not a phase-loss source to filter or delay away.

Fix Attempts — What Was Tried and Ruled Out

- **Narrow band-reject (notch) filter around 5–10 Hz, tested.** Implemented, build-verified, and flown across 0.25–20 Hz bandwidth. Made tracking worse in every stable configuration, one narrow setting $\approx 20\times$ worse. Consistent with the shake being the closed loop's own lightly damped resonant mode rather than exogenous noise: attenuating it in the error signal removes the corrective feedback constraining it instead of removing noise.
- **Lowering k_r , tested.** Also failed, for a different reason: triggered a slower divergence from a violated position/attitude cascade timescale separation, since k_r cannot be reduced independently of the position gains.
- **What the data directly shows:** brushless's raw angular-acceleration signal has a narrowband spectral peak at ≈ 7.22 Hz, and its commanded torque is markedly larger than the other two platforms across all axes (τ_z RMS 2.17 vs. 0.19 (standard) and 0.14 (upgraded) N m — roughly an order of magnitude higher). Both directly measured, not inferred.

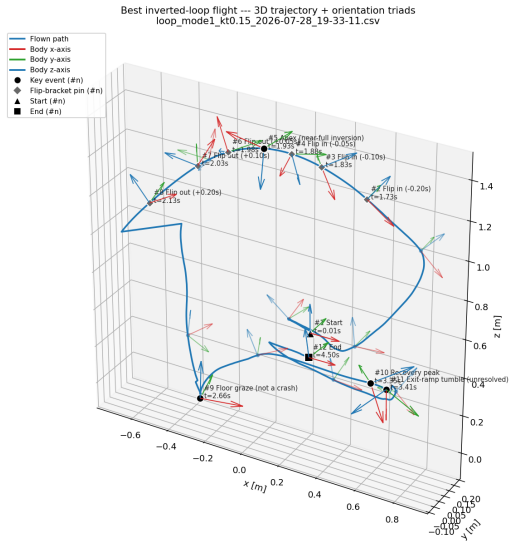
Both targeted fix attempts failed — the resonance was not resolved within this project's scope. Diagnosis is complete and quantified; the fix is genuine future work.

Limitations — Teardrop / Inverted Loop

Status: partial success, bonus work beyond core scope

- **Inverted vertical loop, brushless platform:** repeatable inversion achieved — roll $163\text{--}178^\circ$ at the apex, 3/3 flights, using a separate explicit-attitude planning approach (not the flatness-based methods used elsewhere in this talk).
- Reliable recovery after the flip has not yet been made to work; several fixes were tried and ruled out.
- `teardrop_wide`: crashed at `kt=0.05` on *both* platforms, even under plain geometric control (not just INDI) — unexplained, never root-caused.

Best Inverted-Loop Flight — 3D Trajectory



Summary

- ➊ **Motion planning:** built a pipeline that generates optimal trajectories for the drone — the foundation everything else in this project flies on.
- ➋ **Baseline:** established and tuned a geometric controller on the standard CF2.1 for optimal tracking error, as the reference every other mode is measured against.
- ➌ **INDI:** implemented and deployed full INDI on **three platforms** — standard, thrust-upgraded, and brushless CF21BL.
- ➍ **Comparison:** fair, apples-to-apples evaluation across all four modes — INDI beats geometric on every platform, and brushless beats standard/upgraded on raw tracking (best figure-8 RMSE of all four, 2.45 cm).
- ➎ **Artifacts:** flights across the full trajectory library (figure-8, oval, circle, ...) on multiple platforms, at multiple speeds — the concrete outcome of the project.
- ➏ **Limitations:** brushless "shaky shaky" behavior (two fix attempts tried and ruled out, not resolved) and the inverted-loop maneuver (flip achieved, recovery not yet made to work) — both well-characterized open problems, not unexplained gaps.

Appendix — Motion Planning, In Brief

- ➊ **Waypoints in, smooth path out:** a handful of position (+ yaw) waypoints go in; a piecewise polynomial trajectory that passes through all of them comes out.
- ➋ **Minimum-snap polynomials:** each segment between waypoints is a degree-7/8 polynomial, solved so the whole path is as smooth as possible (continuous up to snap, the 4th derivative of position) — this is what keeps the drone from having to make any sudden, jerky corrections.
- ➌ **Segment timing, two ways:** either give each segment a duration by hand (Mode 0, Mellinger 2012), or give one aggressiveness scalar k_t and let the planner pick timings automatically (Mode 1, Richter et al. 2016) — this is the k_t used throughout every sweep in this talk.
- ➍ **Differential flatness turns a path into a full reference:** position and its derivatives (velocity, acceleration, jerk, snap) plus yaw are enough to solve, in closed form, for everything the controller needs — desired attitude R_d , thrust, body rate ω_d , and angular acceleration $\dot{\omega}_d$ (Faessler, Franchi & Scaramuzza 2018). No attitude has to be authored by hand.
- ➎ **Result:** a complete, physically consistent feedforward reference at every instant in time — not just a position setpoint — exported as polynomial coefficients and evaluated onboard at 500–1000 Hz. Both the geometric and INDI controllers consume the exact same reference.

Appendix — Full INDI, Pseudocode

every control cycle (dt):

- *position loop (outer, full rate)* -

$e_p = p_d - p$; $e_v = v_d - v$

$a_{\text{model}} = (\sum_i k_{t,i} \text{rpm}_i^2) / \text{mass} \cdot R e_z - g e_z$ (*predicted accel, from RPM² thrust model*)

$a_{\text{meas}} = R \cdot \text{accel_body} - g e_z$ (*measured accel, from IMU*)

$a_{\text{indi}} = a_{\text{meas}} - a_{\text{model}} \leftarrow$ *INDI increment: what the model misses*

$f_d = a_d + K_p e_p + K_v e_v + K_i f_e_p + g e_z + a_{\text{indi}}$

$R_d, \omega_d, \dot{\omega}_{\text{des}} = \text{flatness}(f_d \cdot \text{mass}, \text{yaw}_d)$ (*desired attitude, from planning*)

- *attitude loop (inner), the "full INDI" part* -

$e_R = 0.5 \text{vee}(R_d^T R - R^T R_d)$ (*attitude error on SO(3)*)

$e_\omega = \omega - \omega_d$

$\alpha_{\text{ref}} = \dot{\omega}_{\text{des}} - K_r e_R - K_w e_\omega$ (*desired angular accel, Tal & Karaman Eq. 28*)

$\alpha_{\text{meas}} = \text{lowpass}(\text{diff}(\omega))$ (*measured angular accel, from gyro*)

$\tau_{\text{current}} = \text{rpm_to_torque}(\text{m1..m4})$ (*current torque, from motor RPM -- not a model*)

$\Delta\tau = J(\alpha_{\text{ref}} - \alpha_{\text{meas}}) \leftarrow$ *INDI increment: correct only the error*

$\tau = \tau_{\text{current}} + \Delta\tau$

$\text{send}(\text{thrust}, \tau) \rightarrow$ motor mixer

The core idea: instead of computing torque from a dynamics model alone, INDI takes the *currently measured* torque/acceleration (from RPM and gyro) as the baseline and computes only a small correction for the gap between desired and measured — far less sensitive to model error (motor dynamics, drag, mass) than a purely model-based controller.